

Bridges & Boneheads

- **Genre:** 3D platformer “Friendslop”
- **Concept:** Pour traverser d'immenses environnements abstraits et franchir des gouffres mortels, les 2 à 4 joueurs doivent littéralement s'utiliser comme matériaux de construction. La mécanique centrale permet à chaque joueur de passer à volonté d'un état "ragdoll" (léger, élastique, pouvant être attrapé et lancé) à un état "bloc de béton" (lourd, immobile, adhérent). Vous devez donc lancer vos amis au-dessus du vide, leur crier de se figer en plein vol pour créer une plateforme ou un pont, puis sauter dessus. Le chaos naît de la communication : si le joueur qui sert de pilier à tout le groupe annule sa putréfaction par erreur, tout le monde tombe.
- **Public cible:** Groupe d'amis sur Discord, streamers et créateurs de contenu. Public de 20 à 35 ans qui ont déjà des revenus.
- **Plateforme visées:** PC (steam) avec prise en charge de l'intégration Twitch pour que les spectateurs puissent par exemple déclencher des "dégels" surprises.
- **Modèle économique:** Prix bas de 13€99. Accent sur la campagne commerciale en misant sur une campagne d'influence, distribution de clé au streamers, campagnes marketing sur tiktok. Achat de “4 pack” sur steam pour acheter le jeu pour 4 personnes à prix réduit. Jeu volontairement accessible pour les pc de petite config de manière à étendre le public ciblé.

Contraintes :

- **Développement en 3D :** le jeu nécessite la gestion des déplacements, des sauts, des collisions et des interactions physiques entre les personnages. Les environnements seront abstraits et les graphismes stylisés pour limiter la charge de travail du graphiste. Le décor évoquera une vallée montagneuse sombre, avec une ambiance inspirée du « Mordor », tout en conservant des créations originales, sans reprendre directement ses éléments distinctifs. Les graphismes seront proches du « low-poly » pour permettre au graphiste de réaliser l'ensemble des ressources visuelles dans le temps disponible.

- **Deux états physiques** : en mode « ragdoll », le personnage peut être attrapé et lancé. En mode « béton », il conserve sa position et sa posture, même dans les airs, pour servir de plateforme. Les formes des ponts dépendent donc de la posture et de l'assemblage des personnages, sans permettre la création libre de n'importe quelle forme.
- **Multijoueur en ligne de 2 à 4 joueurs** : chaque participant joue depuis son PC. Les déplacements, les collisions et les transformations doivent être synchronisés pour que tous puissent interagir avec les mêmes plateformes. Les déconnexions doivent également être prises en compte, notamment lorsqu'un joueur sert de support aux autres.
- **Progression et réapparition** : le jeu est prévu pour des parties à deux, trois ou quatre joueurs. Les environnements sont adaptés au nombre de participants : plus les joueurs sont nombreux, plus les gouffres à franchir sont importants. La progression est continue, sous la forme d'un long chemin, sans menu de sélection des niveaux. Des points de réapparition fréquents permettent néanmoins de distinguer différentes sections du parcours. Un joueur ayant franchi un gouffre peut harponner ses camarades qui ont servi de pont pour leur permettre de le rejoindre et de poursuivre la progression.
- **Intégration Twitch** : le jeu peut fonctionner avec ou sans connexion à Twitch. Lorsque cette option est activée par le groupe, les spectateurs peuvent intervenir dans la partie en déclenchant des événements destinés à ralentir la progression, comme des « dégels » surprises. Un délai entre deux événements limitera ces interventions pour éviter de bloquer la progression.
- **Défi technique — synchronisation physique** : les deux développeurs devront gérer la synchronisation réseau d'une physique complexe. Les joueurs doivent pouvoir se porter, se lancer et se transformer en supports rigides de manière cohérente sur tous les écrans. Cela nécessite une gestion rigoureuse des états et des collisions, ainsi que des tests dans différentes conditions réseau pour limiter les désynchronisations et les bugs.

SWOT:

Strength:

- **Direction artistique adaptée à une petite équipe** : les graphismes stylisés, proches du low-poly, limitent la charge de travail du seul graphiste.
- **Accessibilité commerciale et technique** : le prix prévu de 13,99 € et l'objectif de fonctionnement sur des PC aux configurations modestes peuvent faciliter l'accès au jeu.
- **Mécaniques propices au partage** : les interactions ragdoll/béton et le harpon peuvent générer des situations chaotiques et amusantes, faciles à présenter dans des clips courts sur les réseaux sociaux.

Weaknesses:

- **Défi technique pour seulement deux développeurs** : synchroniser la physique de quatre joueurs et gérer leurs collisions en temps réel représente une charge de travail importante.
- **Conception des parcours complexe** : adapter les gouffres et les obstacles au nombre de joueurs augmente le travail de création, d'équilibrage et de test.
- **Budget logiciel et d'audit limité** : l'enveloppe de 10 000 € impose de choisir soigneusement les licences, les éventuelles solutions réseau spécialisées et les prestations d'audit.
- **Ressources commerciales limitées** : une seule personne doit gérer plusieurs canaux de communication chronophages, notamment TikTok, les influenceurs et les partenariats.
- **Gestion du parcours continu** : la progression sans sélection de niveaux nécessite d'organiser le chargement des différentes zones pour limiter les interruptions et la consommation de mémoire.

Opportunité:

- **Public susceptible d'acheter le jeu** : les adultes de 20 à 35 ans disposant de revenus constituent une cible potentielle pour un jeu à prix accessible, même si leur intention d'achat reste à vérifier.
- **Présence d'un public pour la coopération** : des jeux comme *Human Fall Flat* et *Chained Together* constituent des références pour positionner notre offre auprès des amateurs de défis entre amis.
- **Possibilités de diffusion sur Twitch** : les streamers qui recherchent des interactions avec leur communauté peuvent être intéressés par les interventions des spectateurs dans le jeu.
- **Formats courts sur les réseaux sociaux** : TikTok et les autres plateformes de vidéos courtes offrent des canaux pour diffuser les moments amusants et faire découvrir le jeu.

Menace:

- **Visibilité incertaine auprès des créateurs de contenu** : si les streamers et les influenceurs relaient peu le jeu, la stratégie commerciale risque de toucher moins de joueurs que prévu.
- **Qualité variable des connexions** : une latence importante ou des pertes de connexion peuvent perturber les interactions physiques, dégrader l'expérience et entraîner des avis négatifs.
- **Apparition de concepts similaires** : des studios disposant de davantage de moyens pourraient développer des mécaniques proches et rendre notre jeu moins distinctif.
- **Concurrence et évolution des goûts** : le jeu doit trouver sa place parmi les titres coopératifs existants. L'intérêt du public pour ce type d'expérience peut également évoluer pendant les deux années de développement.

Choix des technos

ADR-001 — Moteur de jeu

Contexte :

Le studio dispose de deux développeurs, d'un graphiste et d'un commercial pour réaliser le jeu en deux ans. Le budget consacré aux licences logicielles et aux prestations d'audit est de 10 000 €.

Le moteur doit permettre de développer un jeu de plateformes 3D pour PC, avec une physique multijoueur pour deux à quatre joueurs, des graphismes stylisés et une intégration facultative à Twitch. La synchronisation des personnages et les performances sur des configurations modestes sont les principales contraintes techniques.

Décision :

Moteur de jeu: Unity

- Unity propose un environnement adapté au développement 3D et à la publication sur PC.
- Son Asset Store permet de trouver des ressources et des extensions pouvant faciliter certaines intégrations, notamment le réseau ou Twitch. Leur compatibilité, leur maintenance et leur coût devront être vérifiés avant utilisation.
- Les outils d'importation et d'intégration des modèles 3D peuvent faciliter le travail du seul graphiste de l'équipe.
- Le délai de deux ans nous pousse à privilégier un environnement documenté, disposant de ressources d'apprentissage et de composants réutilisables. Un temps de prise en main reste néanmoins à prévoir.
- L'abandon du Runtime Fee supprime cette source de coût, mais ne garantit pas la gratuité du moteur. Nous devons vérifier la licence applicable au studio en fonction de ses revenus et financements, puis intégrer son coût éventuel au budget.

Alternatives écartées:

Unreal Engine :

- **Pourquoi envisagé :** moteur complet pour le développement 3D, proposant des outils de physique et de multijoueur.
- **Avantages :** outils visuels riches et possibilités de rendu avancées.

- **Limites** : pour notre petite équipe, nous estimons que la prise en main et la configuration de cet environnement pourraient demander un investissement important. Ses possibilités graphiques dépassent aussi les besoins de notre direction artistique proche du low-poly.
- **Raison du rejet** : nous privilégions Unity pour organiser la production autour de notre rendu stylisé et de composants réutilisables. Unreal reste techniquement capable de réaliser ce jeu ; nous ne le rejetons pas pour une incompatibilité avec les configurations modestes.

Godot Engine:

- **Pourquoi envisagé** : moteur libre et gratuit, permettant de développer des jeux 3D sans frais de licence du moteur.
- **Avantages** : son utilisation permettrait de préserver une plus grande part des 10 000 € pour les autres licences et les audits techniques.
- **Limites** : Godot possède déjà des fonctionnalités réseau, mais la synchronisation de nos ragdolls, des ponts et du harpon demanderait un développement spécifique. Les extensions adaptées à ces besoins devraient être étudiées.
- **Raison du rejet** : nous privilégions l'écosystème Unity pour rechercher des composants réutilisables et du support sur ces intégrations. Ce choix devra être confirmé par un prototype : il ne signifie pas que Godot imposerait de recréer tout le réseau de zéro, ni que Unity fournirait une solution complète sans travail supplémentaire.

Conséquences:

Les développeurs pourront s'appuyer sur les outils et extensions de Unity pour construire le jeu. Le graphiste disposera d'un environnement permettant d'importer et d'assembler les ressources 3D stylisées.

En contrepartie, l'utilisation d'extensions ajoute des dépendances à maintenir et des risques de compatibilité. L'intégration Twitch et la synchronisation physique nécessiteront toujours du développement et des tests.

Le budget de 10 000 € couvrira les éventuelles licences Unity, les extensions retenues et les prestations d'audit. La campagne marketing devra disposer d'un financement distinct. La solution réseau précise reste à sélectionner après comparaison de ses fonctionnalités, de ses coûts et de son comportement sur un prototype.

Risques et mitigations:

- **Dépendance à une solution réseau externe** : une extension peut présenter des bugs difficiles à analyser ou ne plus être maintenue. Nous vérifierons sa documentation, son suivi et ses possibilités de diagnostic avant de l'adopter.
- **Synchronisation physique insuffisante** : un prototype réalisé en début de projet permettra de tester les lancers, les transformations et le harpon à deux, trois et quatre joueurs, dans différentes conditions réseau. Un audit technique pourra ensuite compléter ces essais avant la production de l'ensemble du parcours.
- **Dépassement du budget** : les coûts des licences et des extensions seront évalués sur les deux années, en conservant une enveloppe pour l'audit. Si les coûts dépassent le budget disponible, le choix devra être réexaminé.
- **Performances insuffisantes** : des tests réguliers sur une configuration PC de référence permettront d'ajuster les graphismes, les collisions et le nombre d'éléments physiques simulés.

ADR-002 — Langage

Contexte

Le moteur retenu est Unity. Les deux développeurs doivent programmer les déplacements, les transformations ragdoll/béton, le harpon et les interactions multijoueurs, dans un délai de deux ans.

Le langage doit faciliter le développement et la maintenance du jeu, tout en permettant de maîtriser les performances sur des configurations PC modestes.

Décision :

Langage retenu : C#.

Pourquoi :

C# est le langage de script principal de Unity. Son intégration au moteur permet de programmer les mécaniques du jeu et d'utiliser les composants proposés par son écosystème.

Il permet d'organiser le code autour de responsabilités distinctes : gestion des personnages, changements d'état, harpon et échanges réseau. Cette organisation doit faciliter les modifications et les tests pendant le développement.

Avantages:

- Une syntaxe structurée et un typage statique qui permettent de détecter certaines erreurs à la compilation.
- Une intégration directe aux outils de développement et de débogage de Unity.
- Un Garbage Collector qui automatise la récupération de la mémoire managée devenue inutilisée, tout en nécessitant une attention aux allocations pour éviter des ralentissements.
- La possibilité de réutiliser des bibliothèques et des composants compatibles avec Unity.

La maîtrise de C# par l'équipe devra être prise en compte dans le planning. Les performances seront vérifiées sur un prototype et par des mesures régulières, plutôt que supposées garanties par le langage ou l'expérience des développeurs.

Alternatives:

C++

- **Pourquoi envisagé** : il permet de développer des composants natifs et de contrôler finement certains traitements coûteux.
- **Avantages** : bonnes possibilités d'optimisation et accès à des bibliothèques natives.
- **Limites** : dans Unity, son utilisation passe notamment par des plugins natifs et ne remplace pas directement les scripts C#. Elle ajoute une chaîne de compilation et des échanges entre code C# et code natif.
- **Raison du rejet** : nous ne retenons pas C++ comme langage principal, afin de limiter la complexité du développement et du débogage. Son utilisation ponctuelle resterait envisageable si des mesures démontrent un besoin précis.

-

Visual scripting:

- **Pourquoi envisagé** : cette approche permet de construire visuellement des comportements et de prototyper rapidement des mécaniques simples.
- **Avantages** : représentation graphique des interactions et possibilité d'ajuster certains comportements sans écrire de code textuel.
- **Limites** : pour notre logique multijoueur, la multiplication des graphes et des connexions pourrait rendre le suivi des changements et la maintenance plus difficiles.
- **Raison du rejet** : nous privilégions C# pour faciliter les revues de code, les tests et l'organisation des interactions complexes. Le Visual Scripting pourrait néanmoins servir à des prototypes ou à des événements simples du décor.

Conséquences:

- L'équipe utilisera un langage directement intégré à Unity pour l'essentiel du gameplay.
- Les composants de l'écosystème pourront être réutilisés, sous réserve de vérifier leur compatibilité et leur qualité.
- Les développeurs devront surveiller les allocations mémoire et les traitements exécutés fréquemment.
- La stabilité du multijoueur dépendra aussi de l'architecture réseau et des tests : le choix de C# ne suffit pas à garantir une synchronisation correcte.

Risques et mitigations

- **Ralentissements liés aux allocations mémoire** : utiliser les outils de profilage pour identifier les allocations fréquentes, puis réutiliser certains objets lorsque cela est pertinent, notamment avec l'Object Pooling.
- **Désynchronisation des états physiques** : définir quelle instance valide les changements d'état et tester les interactions à deux, trois et quatre joueurs, avec différentes latences.
- **Code difficile à maintenir** : séparer les responsabilités et effectuer des revues de code régulières.
- **Temps d'apprentissage sous-estimé** : prévoir une phase de prise en main et un prototype des mécaniques principales dès le début du projet.